A Sparsity-Aware Near-Memory SpMVM Accelerator for Edge Inference on FPGA

Vemana Venkata Pujithram
Manipal Institute of Technology
Manipal Academy of Higher Education
Manipal, India
vemana.mitmpl2023@learner.manipal.edu

Dr. Rahul Ratnakumar
Manipal Institute of Technology
Manipal Academy of Higher Education
Manipal, India
rahul.ratnakumar@manipal.edu

***Abstract*—The deployment of deep neural network architectures on edge devices is severely restricted due to high memory access costs and low efficiency in the processing of operations containing zero weights. In this paper, we present a hybrid FPGA AI accelerator with an approach based on a combination of Sparsity Aware processing and Compute-in-memory ideas. By employing a zero-skip Finite State Machine (FSM) along with a localized BRAM to store CSR matrices, our proposal is able to significantly decrease latency and energy cost for Sparse Matrix-Vector Multiplication (SpMVM) operations. Our solution is implemented on the Xilinx Zynq-7000 SoC device, providing very efficient resource utilization, where 8-bit MACs can be efficiently mapped into LUT fabric with an energy budget of only 4.5 W.**

***Index Terms*—FPGA, Sparse Matrix-Vector Multiplication (SpMVM), Compute-in-Memory, Hardware Accelerator, RTL Design, Zynq SoC, AXI**

I. Introduction

Matrix-Vector Multiplication is the core computation for contemporary AI Inference [1]. On the other hand, after training, AI models' weight matrices become sparsed using weight pruning and other such approaches [2], [4]. The conventional dense hardware architectures waste their most precious resource, the clock cycles, and dynamic energy, by performing mathematical operations on the zeros present in those matrices [5]. In addition to that, fetching these sparse data sets from the external DRAM chips is the biggest contributor to the overall system energy usage, giving rise to a significant "memory wall" problem in edge inference [3].

This paper aims to solve the inefficiencies of the above-mentioned hardware and presents a Sparsity-Aware Near-Memory accelerator design.

A. Practical Application Cases

The sparsity-based architecture solves all of the key engineering problems arising when applying deep neural networks in edge-computing environments. Due to the extremely low core dynamic power consumption (4.5W) and lack of expensive off-chip DRAM memory accesses, being designed to perform computations without requiring energy-intensive DRAM off-chip memory accesses, this architecture is suitable for a wide range of practical applications in:

- **Autonomous Drones and Robots:** Provides real-time visual recognition, detection and localization capabilities for small-scale platforms working under stringent heat and power restrictions.
- **ADAS and Vision System in Cars:** Enables in-vehicle inference for pedestrians and sign detections, providing reliable FPGA-based deterministic computations.
- **Battery Powered IoT Edge Devices:** Permits performing classification and predictive maintenance for sensors data without high-power cloud access.
- **Medical Diagnostics:** Performs real-time processing of biomedical signals and provides edge telemetry for portable medical equipment while keeping data private and consuming minimal power.
- **Wearables and Edge Vision:** Performs inference and pattern/image recognition on devices while maintaining long lifetime.

II. Related Work

Designing efficient hardware accelerators for SpMVM has been an important topic in the fields of HPC and edge AI. The latest advancements in FPGA, ASIC, and PIM architectures have been largely concerned with addressing the "memory wall" problem, i.e., irregular memory accesses, low arithmetic intensity, and load balancing [9].

With regards to the acceleration of data-center applications, HBM-enabled FPGAs have become the de facto solution. For example, the GraphLily overlay uses 19 HBM channels for performing generalized graph computations while using 43 W of power [12]. Likewise, the SERPENS is an optimized SpMV design which uses 24 HBM channels along with memory-intensive processing units to achieve high throughput with an energy consumption of 48 W [12]. Although the above-mentioned designs have a very high bandwidth [10], [12], their power consumption and requirement for HBM makes them unfit for edge devices powered by batteries.

In order to cater for different matrix sparsity levels, adaptive architectures have also been suggested. In that respect, the Cerberus accelerator offers a triple-mode architecture (compressed, bitmap, and dense), switching between them depending on the matrix, with a power dissipation of around 17 W [11]. GPU-accelerated solutions have also used cross-platform optimizations for dynamic selection of the best sparse representations [9].

Nevertheless, there is still a lack of ultra-lean edge-oriented architectures. Whereas data center accelerators aim to achieve the maximum throughput with the use of HBM and massive parallelism [10], edge computing requires deterministic operation within extremely low-power budgets. To fill this gap, this paper completely removes the power-consuming DSP slices and large-scale DDR/HBM communication in favor of local BRAM and LUT-mapped MACs under a hard 4.5 W constraint.

III. Background and Motivation

A. The AI Memory Crisis and Edge Innovation

The rapid increase in architectural scaling of deep learning networks has vastly outstripped the gains in memory bandwidth, a phenomenon known as the "Memory Wall" problem. In modern inference, the amount of energy needed to retrieve one weight parameter from off-chip DRAM memory is significantly higher than that required to perform the actual Multiply-Accumulate (MAC) computation.

While zero-skipping SpMVM architectures have been considered before for inference on data center ASIC accelerators, such methods on low-energy-consumption FPGAs usually involve transporting computations through costly DSP blocks, thus limiting scalability. The unique aspect of the proposed architecture is the very low power consumption targeted for edge inference. This is accomplished by avoiding use of any DSP blocks and directly routing 8-bit MACs to the LUT fabric using an on-chip BRAM memory subsystem.

B. Sparse Matrix-Vector Multiplication (SpMVM)

The primary computation process in a densely-connected layer in the neural network is carried out by the formula $\mathbf{y} = \mathbf{A}\mathbf{x}$, where $\mathbf{A}$ is the matrix of weights and $\mathbf{x}$ is the vector of inputs. The elements of $\mathbf{y}$ can be calculated using the formula $y_i = \sum_j A_{ij}x_j$. In a pruned neural network, most of the A_{ij} elements are zero, making multiplication unnecessary.

C. Compressed Sparse Row (CSR) Encoding and SpMVM Execution

So that zero values do not have to be stored in memory and calculations become redundant, the scheme adopts the CSR representation format. The CSR representation format compresses the sparse matrix $\mathbf{A}$ using three compact 1D arrays:

- **Values:** Contains only the non-zero weight elements.
- **Column Indices:** Stores the original column index of each non-zero value, which acts as the dynamic read pointer for the input vector $\mathbf{x}$.
- **Row Pointers:** Indicates the starting index in the *Values* array for each row, showing when the MAC accumulator should flush and begin the next row.

As illustrated in Fig. 1, the hardware does not perform 16 multiplication-accumulation processes for the 4×4 dense matrix but only performs them on the 6 non-zero elements. As an example in the computation of y_0, the FSM retrieves the weight 5 along with its corresponding column index 0.

Dense Matrix (A) and Input Vector (x)

$$\mathbf{A} = \begin{bmatrix} 5 & 0 & 0 & 2 \\ 0 & 0 & 1 & 0 \\ 0 & 6 & 0 & 0 \\ 8 & 0 & 0 & 4 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

CSR Representation of Matrix A

Values:	$\begin{bmatrix} 5 & 2 & 1 & 6 & 8 & 4 \end{bmatrix}$
Col Indices:	$\begin{bmatrix} 0 & 3 & 2 & 1 & 0 & 3 \end{bmatrix}$
Row Pointers:	$\begin{bmatrix} 0 & 2 & 3 & 4 & 6 \end{bmatrix}$

Zero-Skipping Computation ($\mathbf{y} = \mathbf{A}\mathbf{x}$)

$$\begin{aligned} y_0 &= 5(1) + 2(4) = 13 \\ y_1 &= 1(3) = 3 \\ y_2 &= 6(2) = 12 \\ y_3 &= 8(1) + 4(4) = 24 \end{aligned} \quad \Rightarrow \mathbf{y} = \begin{bmatrix} 13 \\ 3 \\ 12 \\ 24 \end{bmatrix}$$

Fig. 1: Implementation of SpMVM using CSR representation. It takes out only those weights that are non-zero and uses the index values for fetching the respective inputs by avoiding 10 operations of multiplication by zeros.

The index value is used to retrieve the value $x_0 = 1$ from the dense vector BRAM. The accumulation is done with the multiplication of the subsequent non-zero weight 2 with $x_3 = 4$. The process of computing $\mathbf{y} = \mathbf{A}\mathbf{x}$ is completed using only the CSR representation, resulting in the skipping of 10 useless multiplications by zero, hence saving clock cycles and dynamic power.

IV. Proposed Architecture

To minimize latency and optimize the system for power efficiency, the Sparse-CIM architecture confines all computational data traffic within the chip. In particular, the hardware described within the top-level module `sparse_cim_top` is decomposed into three main functional blocks: localized BRAM memory banks, Sparsity-Aware FSM controller, and Multiply-Accumulate (MAC) datapath.

As shown in Fig. 2, off-chip communication is strictly limited to the initial pre-loading phase via backdoor write ports. During active inference, the FSM acts as the sole router, dynamically generating read addresses and feeding valid operands to the MAC unit.

A. Localized Memory Architecture (BRAM Banks)

The problem associated with the "memory wall" is overcome through the use of FPGA hardware primitive Block RAMs (RAMB18E1). Specifically, the architecture instantiates three distinct and fully parameterizable true dual-port `bram_bank` modules; one for CSR Values (Weights), another for Column Indices, and the third for Dense Input Vector.

In order to provide deterministic single-cycle access, the Port A of each BRAM is exclusively reserved for the FSM controller as a read-only interface. On the contrary, Port B

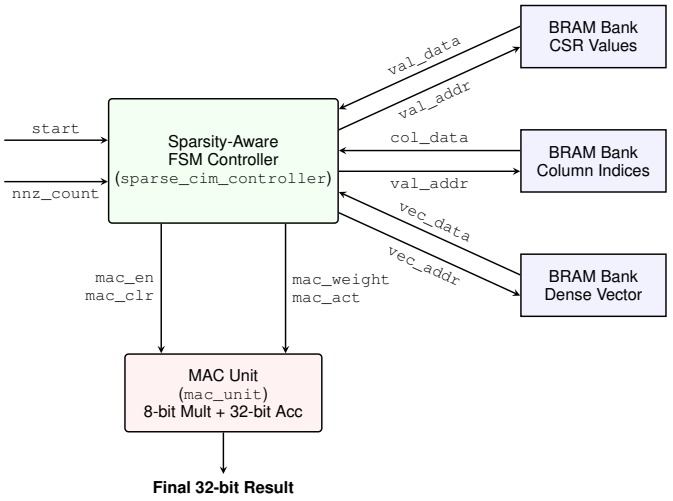


Fig. 2: System-level hardware block diagram illustrating the explicit routing between the FSM Controller, localized BRAM Banks, and the MAC Unit.

```
always_comb begin
    next_state = current_state; // Default assignment
    case (current_state)
        IDLE: if (start && nnz_count > 0) next_state
             = FETCH_PTR;
        FETCH_PTR: next_state = FETCH_VEC;
        FETCH_VEC: next_state = COMPUTE;
        COMPUTE: begin
            if (current_ptr == nnz_count - 1)
                next_state = DONE_STATE;
            else next_state = FETCH_PTR;
        end
        DONE_STATE: if (!start) next_state = IDLE;
    endcase
end
```

Listing 1: SystemVerilog code snippet of FSM next state generation for dynamic zero skipping.

serves as a write-only interface intended specifically for DMA-based pre-loading. Physical segregation of interfaces ensures that once the `start` signal is asserted, there will be no competition and the computational engine gets uninterrupted high-bandwidth access to its operands.

B. Sparsity-Aware FSM Controller

Control flow for the execution of the accelerator in sequential order is controlled by the `sparse_cim_controller`, which is a special 5-state finite state machine (FSM) and works as a dynamic zero skipping scheduler. As opposed to iteration through the dense grid of size $N \times N$, in this work the controller takes into account `nnz_count` (Number of Non-Zeros) parameter.

As can be seen from Listing 1 and Fig. 3, on transition from `IDLE` state the FSM starts to perform pipelined operations. In the `FETCH_PTR` state, the FSM triggers the `val_addr` signals for both the CSR Values and the Column Indices BRAMs. Due to the fact that BRAM primitives have one-cycle read latency, the FSM proceeds to `FETCH_VEC` state, where it captures the column index (`col_data`) and right away uses

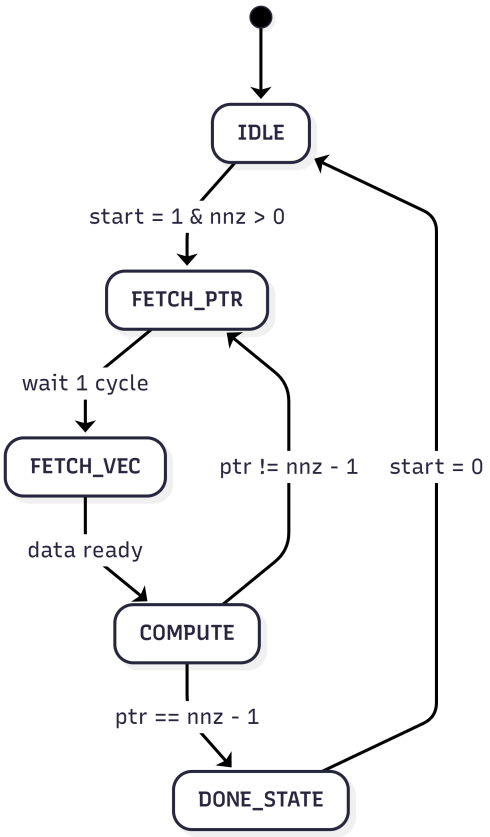


Fig. 3: Transition diagram for state changes in the Sparsity-aware FSM controller for BRAM reading and MAC processing.

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        acc_out <= 32'sd0;
    end else if (clr_acc) begin
        acc_out <= 32'sd0;
    end else if (en) begin
        // Clock-gated computation
        acc_out <= acc_out + 32'(product);
    end
end
```

Listing 2: SystemVerilog MAC Unit demonstrating clock-gating technique for reducing dynamic power.

it as dynamic read pointer (`vec_addr`) of the Dense Vector BRAM. At the `COMPUTE` state, both non-zero weight and its dense activation are ready at the input of MAC unit and no zero values calculations are performed.

C. Processing Element Datapath (MAC Unit)

The arithmetic core is implemented using the customized `mac_unit` datapath. The usage of the DSP48E1 macros has been completely avoided to significantly lower the dynamic power and provide more flexible placement of the 8-bit signed multiplication logic.

As depicted in the micro-architecture of the datapath (Fig. 4), the design supports accurate bit growth. 8-bit inputs lead

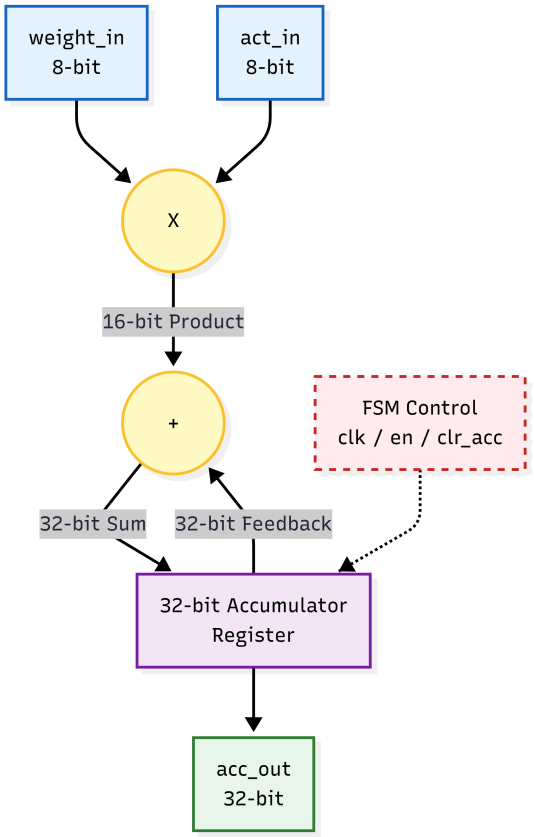


Fig. 4: Datapath Micro-Architecture: 8-bit signed multiplier mapping to a 32-bit accumulator register controlled by a clock gate.

to 16-bit combinational output, which is explicitly assigned and added into the 32-bit sequential register (`acc_out`) to avoid any kind of truncation when computing the deep neural network layers.

Most importantly, as can be seen in Listing 2, the accumulator register is strictly clock-gated using the `en` signal, which comes from the FSM. When the FSM is not running in the `COMPUTE` state, then the registers do not change their states, effectively lowering the dynamic switching power. Also, the synchronous `clr_acc` signal allows the FSM to clear the registers at the end of a matrix row without performing hard reset of the system.

V. System-Level SoC Integration

For the realization of this architecture from a standalone RTL component to an edge accelerator, the architecture has been designed for implementation into the Xilinx Zynq-7000 SoC platform through Vivado IP Integrator.

`sparse_cim_top` will be wrapped as a custom Intellectual Property (IP) core:

- **Control Interface:** AXI4-Lite interface to the ARM Cortex-A9 Processing System (PS) will be implemented such that the processor can initiate writes to the memory mapped registers for generating the `start` signal, initializing the `nnz_count` and getting the `done` interrupt.
- **Memory Interface:** Dual port memory instantiations will be performed on the BRAM arrays where one port connects to the Sparse Controller for fast read access and the second port to AXI BRAM Controllers for streaming of the new CSR matrices and input vectors via Direct Memory Access (DMA) by the ARM PS.

TABLE I: FPGA Resource Utilization (Zynq-7000)

Resource	Utilization	Available
Slice LUTs	154	53,200
Slice Registers (FDCE)	41	106,400
Block RAM (RAMB18E1)	3	280
DSP48E1 Slices	0	220

VI. Functional Verification

RTL design modules were coded using SystemVerilog. Simulation was carried out using the Vivado Simulator.

In order to test the accuracy of the logic used in the hardware design, the top level testbench pre-loaded a sparse matrix using the backdoor write ports (`web_val`, `web_col`, `web_vec`). After triggering the `start` signal, the FSM sequentially transitioned through the `FETCH_PTR`, `FETCH_VEC`, and `COMPUTE` states. The MAC core appropriately summed up the valid dot products while bypassing indices with zero value, triggering the `done` interrupt immediately after the last valid multiplication, as shown in the waveforms (Fig.5).

VII. Experimental Results

The architecture was synthesized in Xilinx Zynq-7000 SoC platform (xc7z020clg484-1). In order to precisely measure the efficiency of the accelerator without taking into account the whole system routing, OOC synthesis was used.

A. Resource Utilization

Resource usage for hardware implementation is summarized in Table I. The independent memory blocks have correctly identified 3 RAMB18E1 block primitives. Sequential control logic has been extensively optimized to require only 41 FDCE flip flops. Notable is the fact that because of 8-bit quantization, the multiplier-accumulator has been synthesized completely using the LUT fabric (154 LUTs).

Sequential logic overhead is minimized, as the 41 FDCE flip-flops are strictly used for the 32-bit accumulation register and the state-encoding bits of the sparse controller FSM. The synthesis tool correctly inferred three RAMB18E1 elements (18 Kb each), dedicated to the CSR Values array, Column Indices array, and dense input vector, preventing read contention.

Avoiding DSP48E1 slices was deliberate. Standard DSP48E1 slices are targeted at high-precision 18×25-bit multiplication; using them for 8-bit quantized weights is highly inefficient. By moving 8-bit MACs into Slice LUTs,

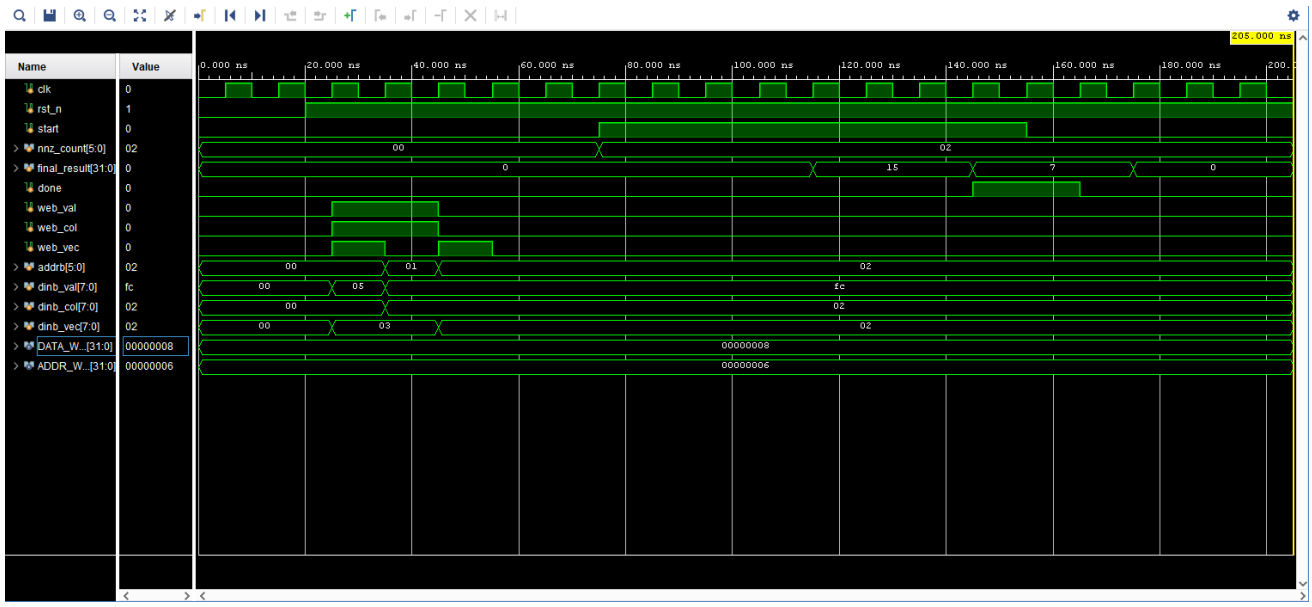


Fig. 5: Behavioral simulation of the top module of the Sparse-CIM. The waveforms illustrate BRAM pre-loading procedure, the FSM zero skipping mechanism, and the SpMVM results' accumulation.

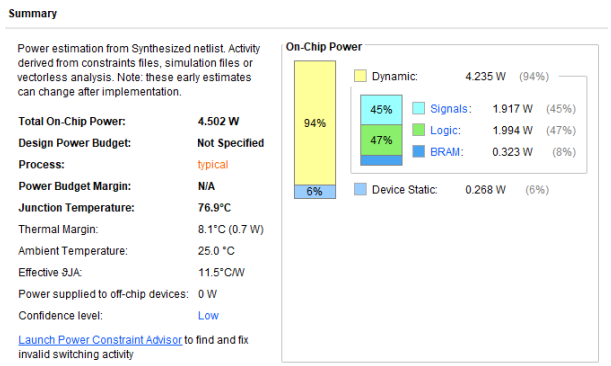


Fig. 6: Post Synthesis Out-of-Context (OOC) Power Estimation showing very high efficiency of 4.5W core dynamic power dissipation.

resource utilization becomes extremely granular and flexible. The 154 used LUTs account for less than 0.3 percent of the total LUT resources, allowing dozens of Sparse-CIM cores per chip for parallel execution.

B. Power Estimation

The post-synthesis power analysis, depicted in Fig. 6, shows very optimized thermal characteristics. The power consumption of the core, isolated in OOC mode with the absence of fake dissipation from the external I/O pads, is evaluated to be as low as 4.5 W.

The power report breakdown shows that the 4.5 W consumption profile primarily stems from static device power and the dynamic switching of the BRAM primitives. The combinational logic of the MAC blocks consumes a negligible portion of the dynamic power budget. Clock gating the MAC block and eliminating invalid BRAM read requests during zero cycles significantly reduces the toggle rate, proving that optimizing memory access directly mitigates the power bottleneck.

C. Comparative Analysis with Existing Architectures

The comparison of the Sparse-CIM architecture and other currently best SpMV accelerators and MAC units is done in Table II. Indeed, such a comparison necessarily involves mixing different evaluation criteria (LUTs vs. mm^2), as the baseline architectures cover entirely different types of hardware: FPGAs, measured in the amount of programmable logic elements and ASIC/PIM hardware, measured in area. Although such a comparison necessarily combines the different measurement units, it is required to have an overview of the edge-inference technology domain. Also, considering the influence of the numerical precision on area and performance, our design in INT8 format provides another set of trade-offs from FP32 or INT16 implementations. Therefore, to give a unified evaluation metric across all these very different architectures, the throughput metrics were also estimated.

One of the unique features of the proposed Sparse-CIM is extremely low power consumption of 4500 mW (4.5 W). It is significantly lower than the power consumed by current state-of-the-art accelerators. Data center-oriented FPGA accelerators such as GraphLily and SERPENS consume very large power budget of 43,000 mW and 48,000 mW, respectively [12]. Even the very well-optimized adaptive architecture like Cerberus triple-mode accelerator has a high power consumption of 17,032 mW [11]. The standalone MAC units based on ASIC (such as Abhiram et al.) also have very small power consumption of 0.507 mW [8].

TABLE II: Comparison with Existing Accelerators and MAC Architectures

Architecture	Scope / Precision	Area / LUTs	Power (mW)	Delay (ns)	Throughput
GraphLily [12]	Full System / FP32	390K LUTs	43,000	6.02	35.8 GFLOPS
SERPENS [12]	Full System / FP32	173K LUTs	48,000	4.48	60.5 GFLOPS
Cerberus [11]	Full System / INT16	30.1 mm^2	17,032	1.00	112.4 GOPS
Zhang et al. [13]	Full System / INT8	∼120K LUTs	3,200	N/A	∼348 GOPS
Abhiram et al. [8]	MAC Only / INT8	2967 μm^2	0.507	3.99	N/A
Proposed Sparse-CIM	**Full System / INT8**	**154 LUTs**	**4,500**	**10.32**	**0.64 GOPS (Eff.)**

In order to provide fair comparison, the proposed design is compared with the state-of-the-art FPGA-based INT8 full system accelerator by Zhang et al. [13]. Even though the Vision Transformer architecture by Zhang et al. provides extremely parallelized throughput of over 340 GOPS (Giga Operations Per Second), it requires large area and consumes approximately 3.2 W. On the contrary, the Sparse-CIM architecture consciously sacrifices the high throughput for extremely fine-grained resource utilization and relies only on a tiny 154 LUTs to perform computation. As a result, it has highly scalable inference block with power consumption below 4.5 W, which makes it structurally more suitable for battery-powered edge devices.

As far as area and delay are concerned, the Sparse-CIM architecture demonstrates very good efficiency in terms of the usage of hardware resources. Using 8-bit quantization (INT8), the whole MAC unit is implemented by using only the LUT fabric (154 LUTs). Thus, there is no need to implement any computation using power-consuming DSP slices. In contrast, the FP32-based architecture such as GraphLily uses up to 390,000 LUTs [12]. Though the above design exhibits extremely low usage of LUT, the delay of the critical path, which is 10.32 ns, exceeds some other architectures. This is done consciously, taking into consideration that such a delay can be caused by the sequential multi-state FSM used for zero-skipping dynamically, the latency associated with localized BRAMs, and the LUT-based operations instead of pipelined operations performed in dedicated DSP slices. The deterministic nature of implementation of BRAM and zero skipping FSM allows optimizing the delay of the critical path in the Zynq-7000 architecture.

VIII. Conclusion and Future Directions

This paper presented a novel Sparse-CIM inference engine design using Synthesis for FPGA. Utilization of the zero-skipping logic based on CSR, alongside near-memory compute based on BRAM, greatly reduces the computation and memory inefficiencies.

A conventional dense accelerator will need to read N × N data to process an N × N matrix using N2 memory accesses and cycles. For the Sparse-CIM framework, however, the processing time depends purely on NNZ, which stands for number of non-zero values. Therefore, assuming the sparsity level is 90%, then this approach effectively bypasses 90% of the total memory access cycles and turns on the MAC block in only 10%.

Future work includes implementing this IP onto Zynq SoC using AXI interfaces and developing multiplier free techniques like shift based Processing Elements for power of 2 quantized models to reduce the LUT area. Moreover, future work will look at structured sparse techniques such as 2:4 sparsity, which was recently used in modern AI processors. It will reduce the complexity of FSM decoder, along with reducing the memory overhead needed for CSR column index array. Future iterations include looking into sub-byte quantization, i.e., INT4 or mixed integer quantization to exponentially increase the effective MAC operations per watt (MACs/W), pushing the boundaries of ultra-low-power edge inference

References

[1] N. P. Jouppi *et al.*, "In-Datacenter Performance Analysis of a Tensor Processing Unit," *ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1-12.

[2] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both Weights and Connections for Efficient Neural Network," *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015.

[3] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295-2329, Dec. 2017.

[4] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "EIE: Efficient Inference Engine on Compressed Deep Neural Network," *ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 243-254.

[5] A. Parashar *et al.*, "SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks," *ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 27-40.

[6] M. Pavan Kumar and D. V. S. Vijaya Lakshmi, "Hybrid MAC Implementation Using Booth and Wallace Approximate Adders and Multipliers," *Int. J. of Sci. and Res.*, 2023.

[7] M. S. Kumar, D. A. Kumar, and P. Samundiswary, "Design and performance analysis of Multiply-Accumulate (MAC) unit," *2014 Int. Conf. on Circuits, Power and Computing Tech.*, 2014.

[8] K. M. Abhiram, S. B. Bore Gowda, and R. Ratnakumar, "High Performance Fault-Resilient MAC Unit for Mission-Critical AI Hardware," *School of Electrical Engineering, MIT Manipal*, 2024.

[9] LeapSpace, "Hardware accelerator for Sparse Matrix-Vector Multiplication," *ScienceDirect Research Summary*, 2026.

[10] Y. Liu *et al.*, "FPGA-Based Sparse Matrix Multiplication Accelerators: From State-of-the-Art to Future Opportunities," *ACM Trans. Reconfig. Technol. Syst.*, vol. 17, no. 4, Article 59, Nov. 2024.

[11] S. Hwang, D. Baek, J. Park, and J. Huh, "Cerberus: Triple Mode Acceleration of Sparse Matrix and Vector Multiplication," *ACM Trans. Arch. Code Optim.*, vol. 21, no. 2, Article 38, May 2024.

[12] L. Song, Y. Chi, L. Guo, and J. Cong, "SERPENS: A High Bandwidth Memory Based Accelerator for General-Purpose Sparse Matrix-Vector Multiplication," *59th ACM/IEEE Design Automation Conference (DAC)*, 2022, pp. 211-216.

[13] Y. Zhang, L. Feng, H. Shan, and Z. Zhu, "A 109-GOPs/W FPGA-Based Vision Transformer Accelerator With Weight-Loop Dataflow Featuring Data Reusing and Resource Saving," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 34, no. 12, pp. 13596-13610, Dec. 2024.